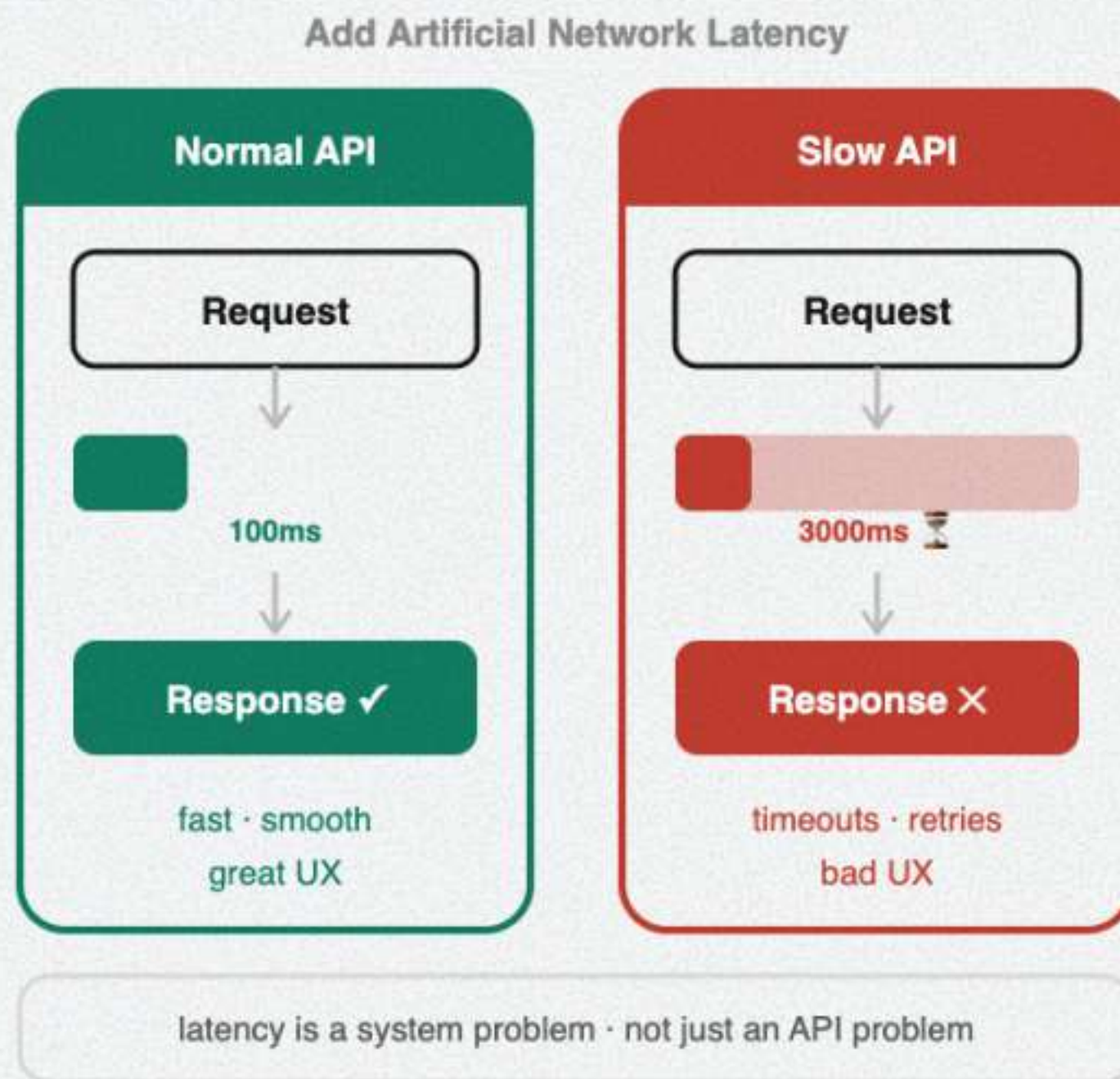


Experiments Every Developer Should Try Once

Break your own code. Learn what happens next.

Add Artificial Network Latency

Make your local API intentionally slow.



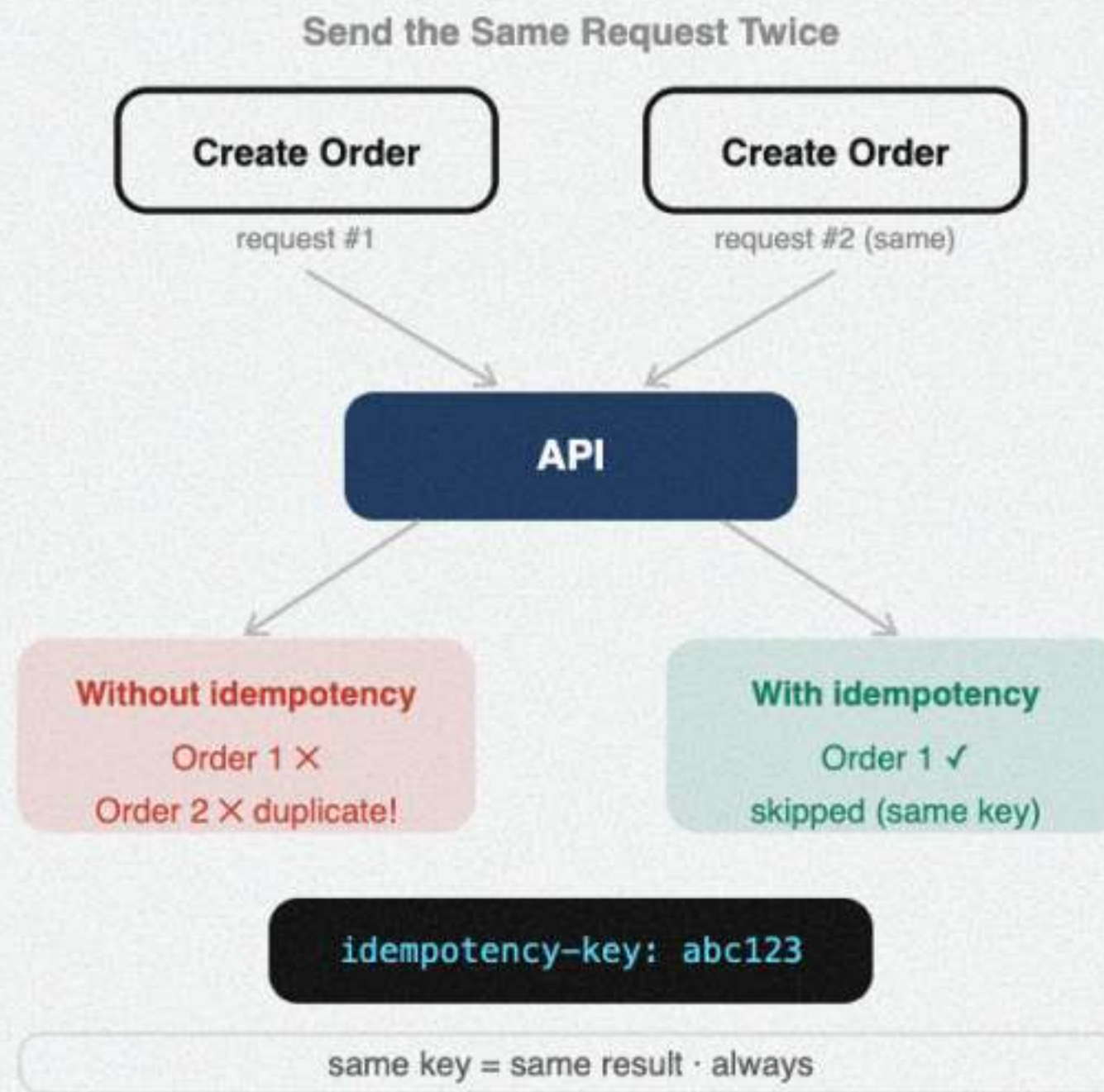
Then observe:

- Loading states
- Response time
- Timeouts
- Retry behavior
- User experience

You'll understand why latency is a system problem, not just an API problem.

Send the Same Request Twice

Take an operation like:



and send the same request twice.

Then ask:

Did my application create one order or two?

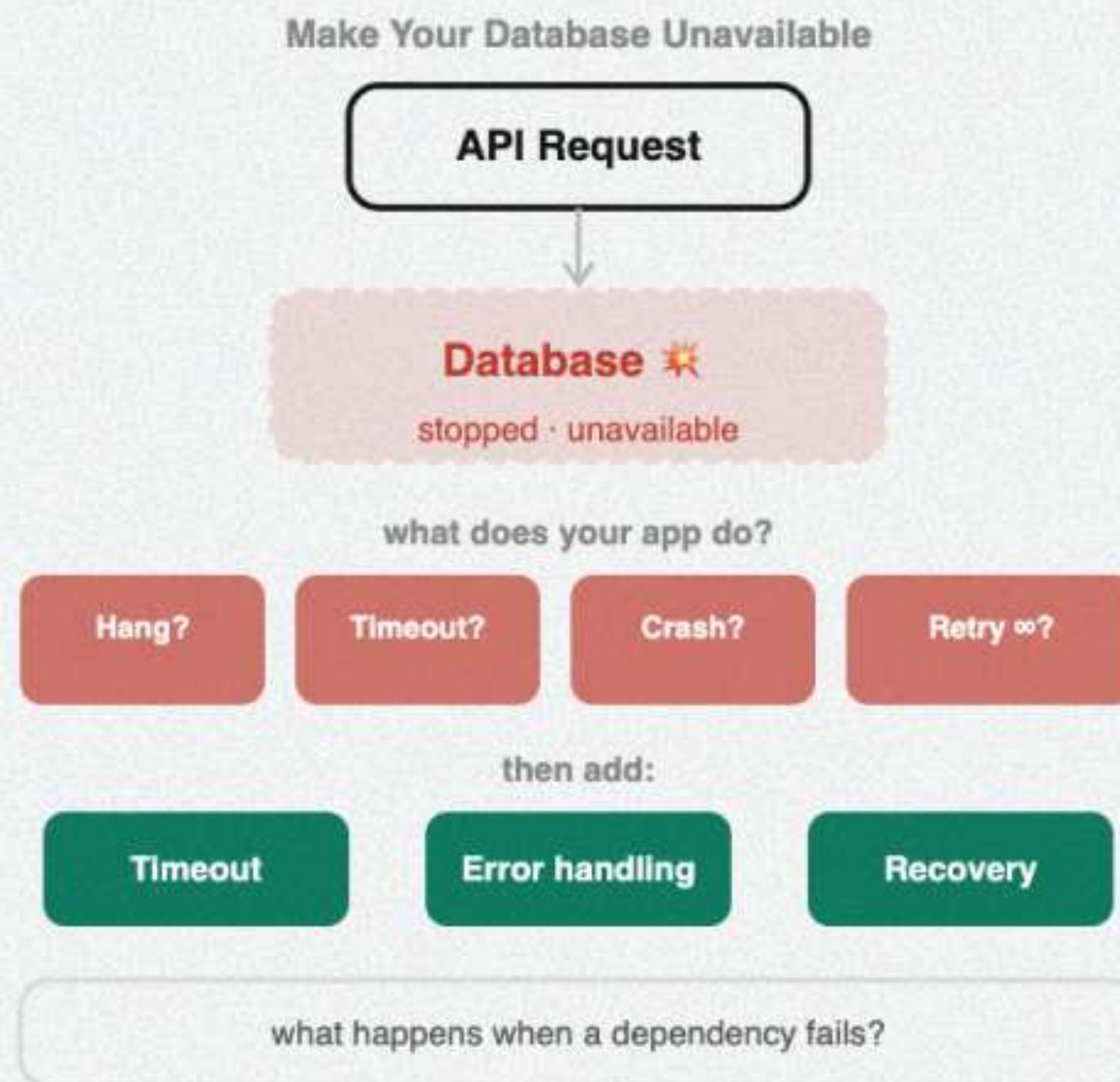
Experiment with:

- Duplicate submissions
- Network retries
- Idempotency keys
- Unique constraints

You'll understand why idempotency matters in real systems.

Make Your Database Unavailable

Only in a local/test environment.



Stop your database and make an API request.

Watch what happens.

Does your application:

- Hang?
- Timeout?
- Return a useful error?
- Crash?
- Retry forever?

Then add

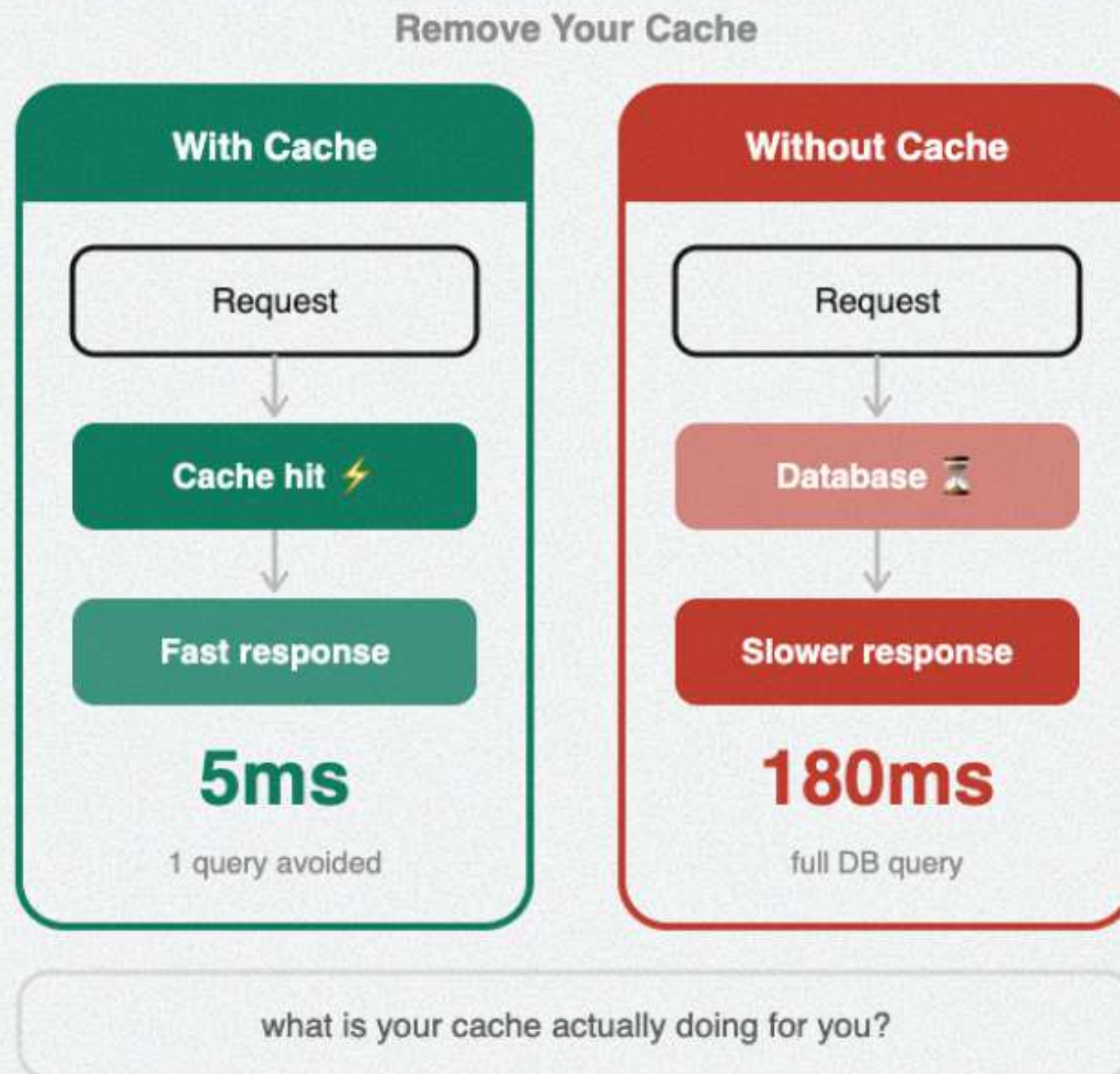
Timeout → Error handling → Recovery

You'll understand

What your application actually does when a dependency fails.

Remove Your Cache

Run the same request:



Then compare:

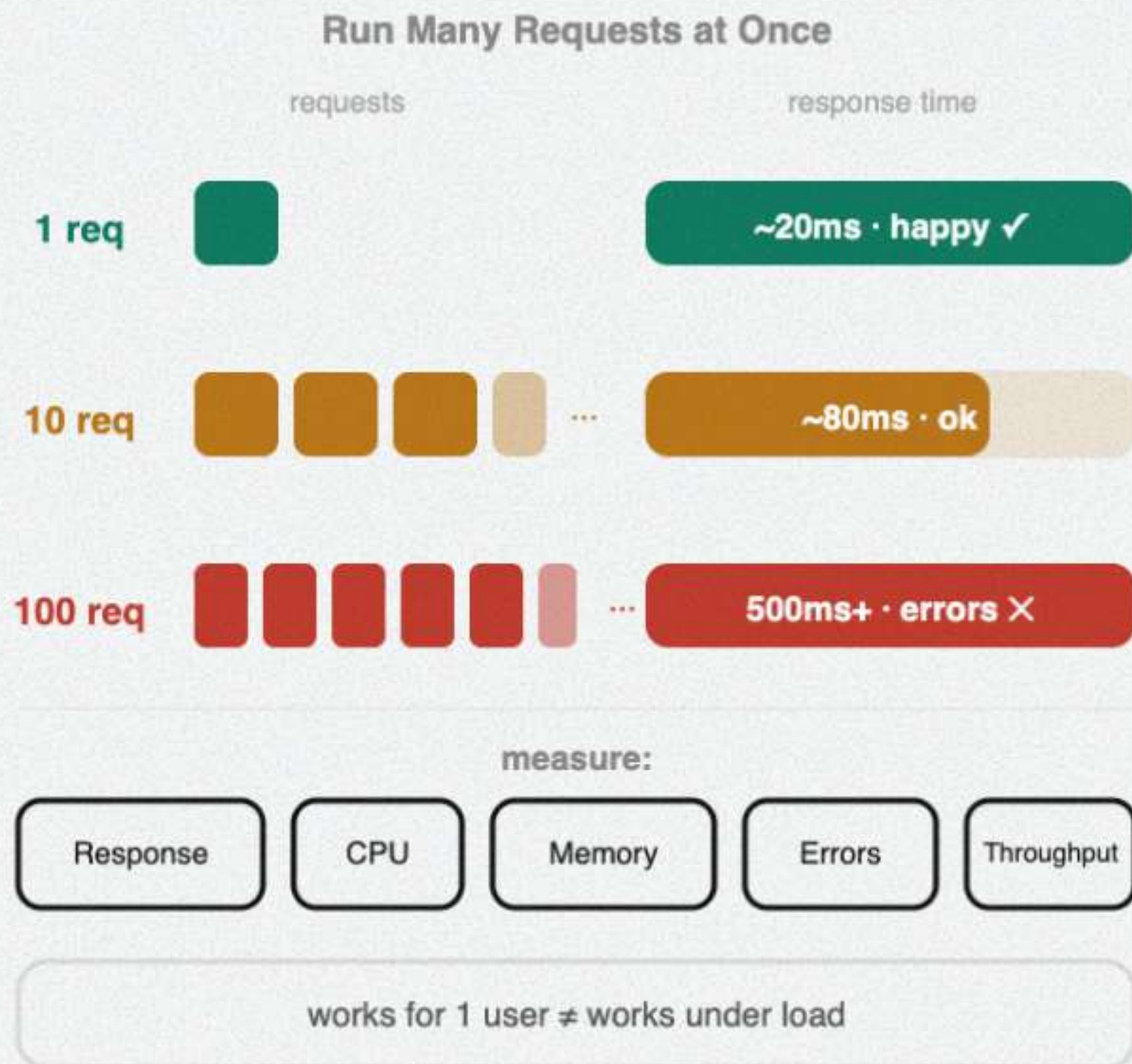
- Response time
- Database load
- Number of queries

You'll understand

What your cache is actually doing for your system.

Run Many Requests at Once

Send concurrent requests to your local application.



Measure:

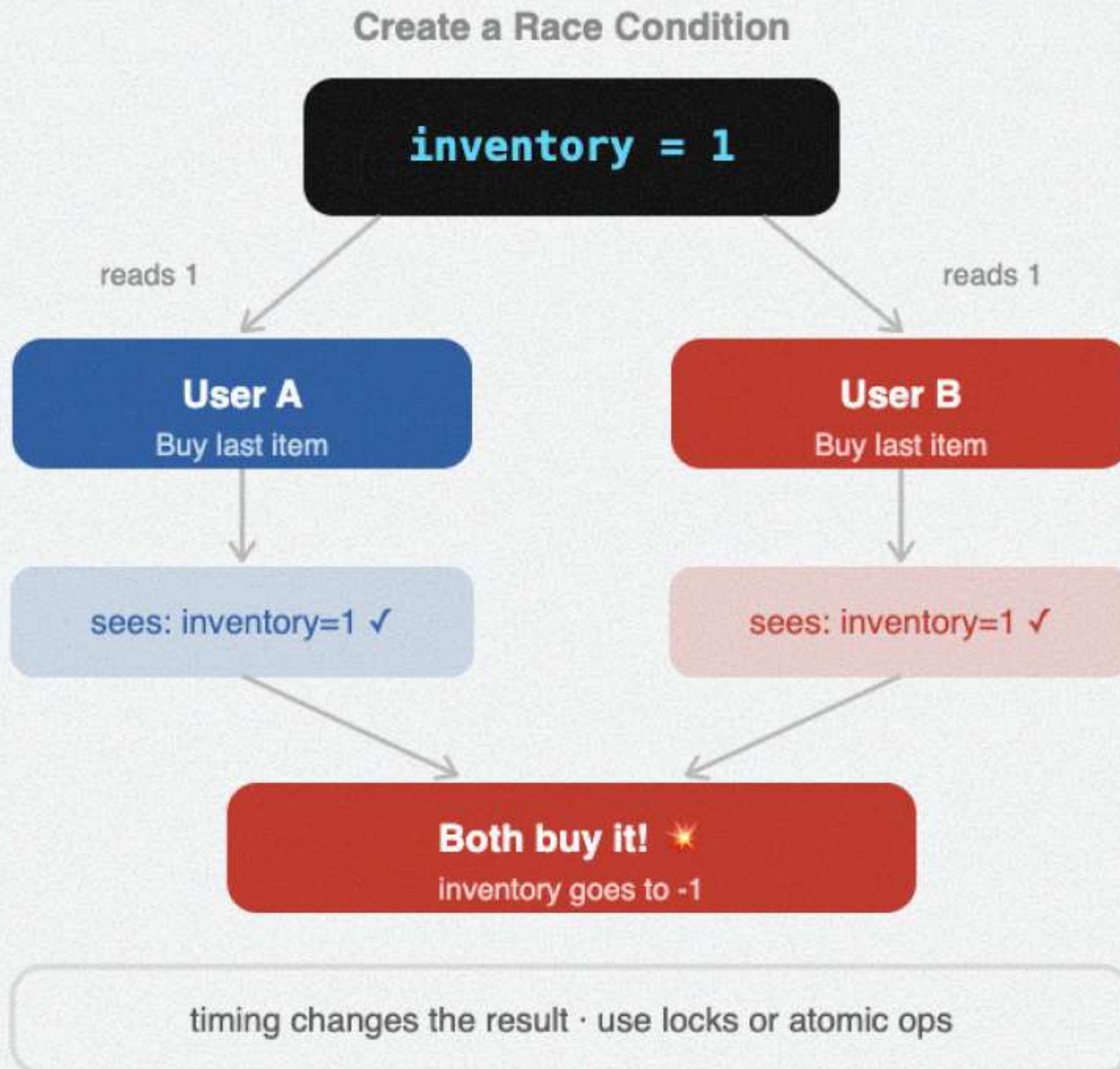
- Response time
- CPU
- Memory
- Errors
- Throughput

You'll understand

Why software that works for one user can behave very differently under load.

Create a Race Condition

Create a shared value:

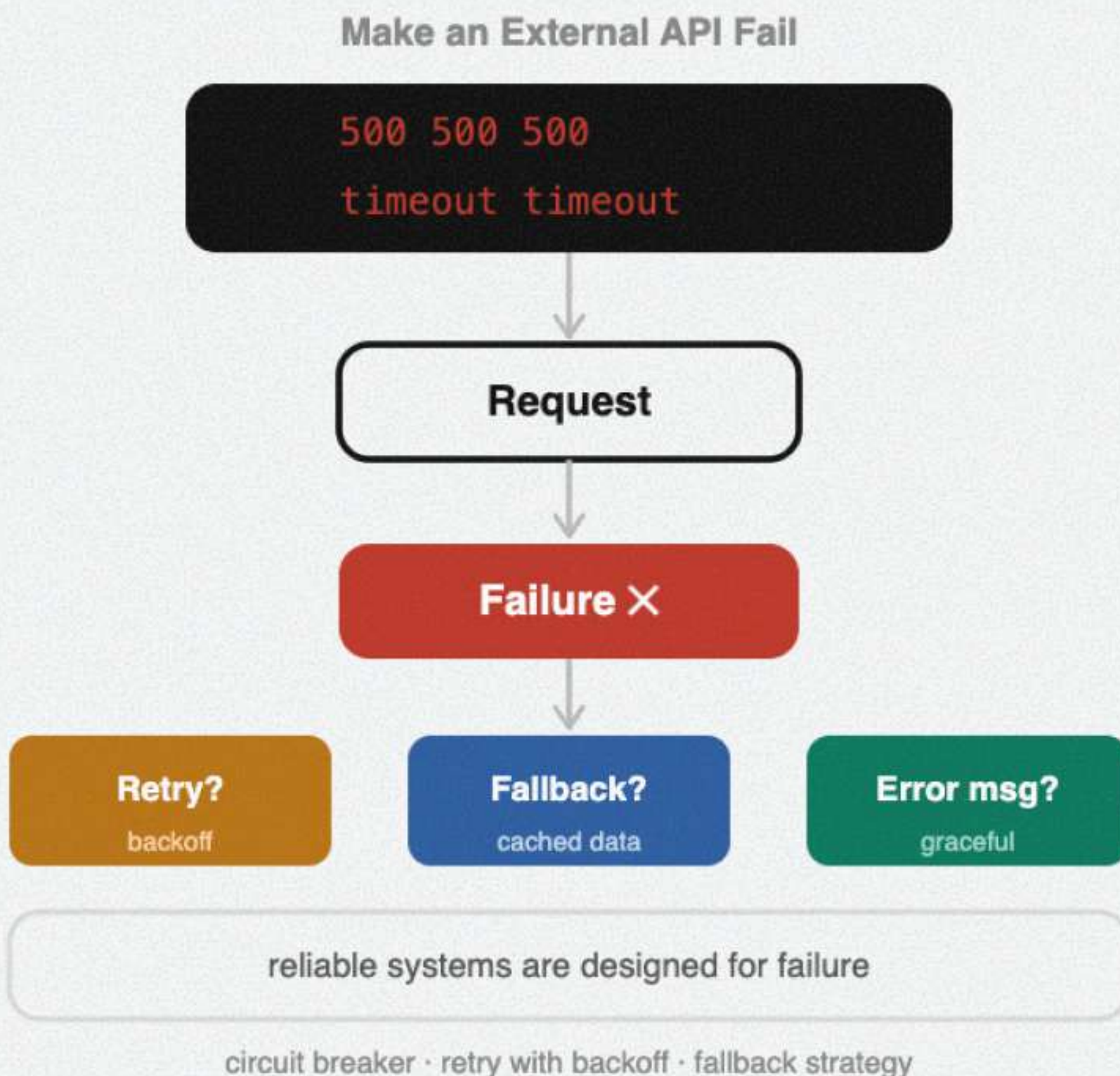


You'll understand

Why timing can change the result of your program.

Make an External API Fail

Use a mock service or local endpoint that intentionally returns:



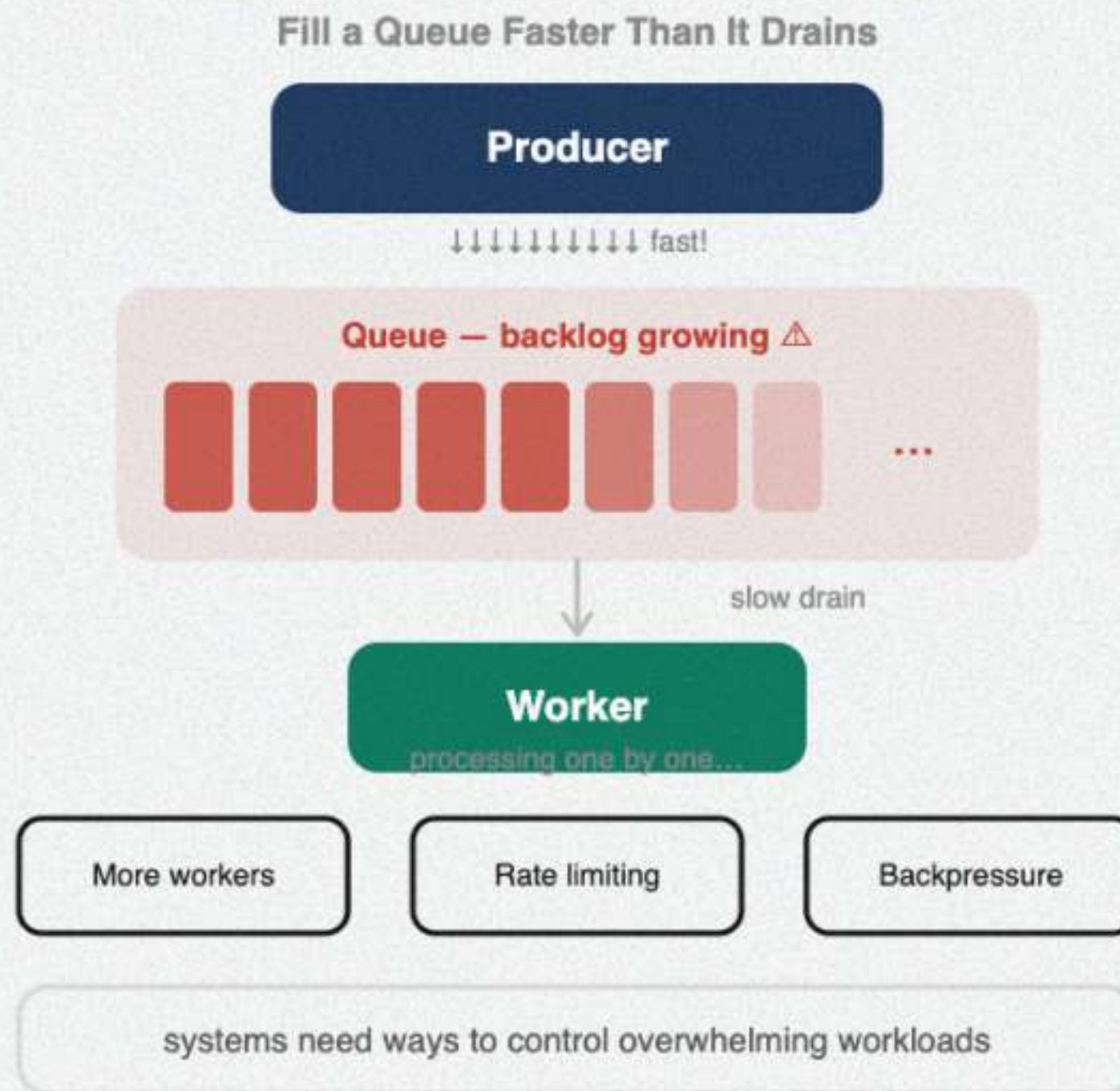
You'll understand

Why reliable systems are designed for failure.

Fill a Queue Faster Than It Can Process

Create a small local queue.

Make producers generate work faster than workers can process it.



Watch the backlog grow.

Then experiment with:

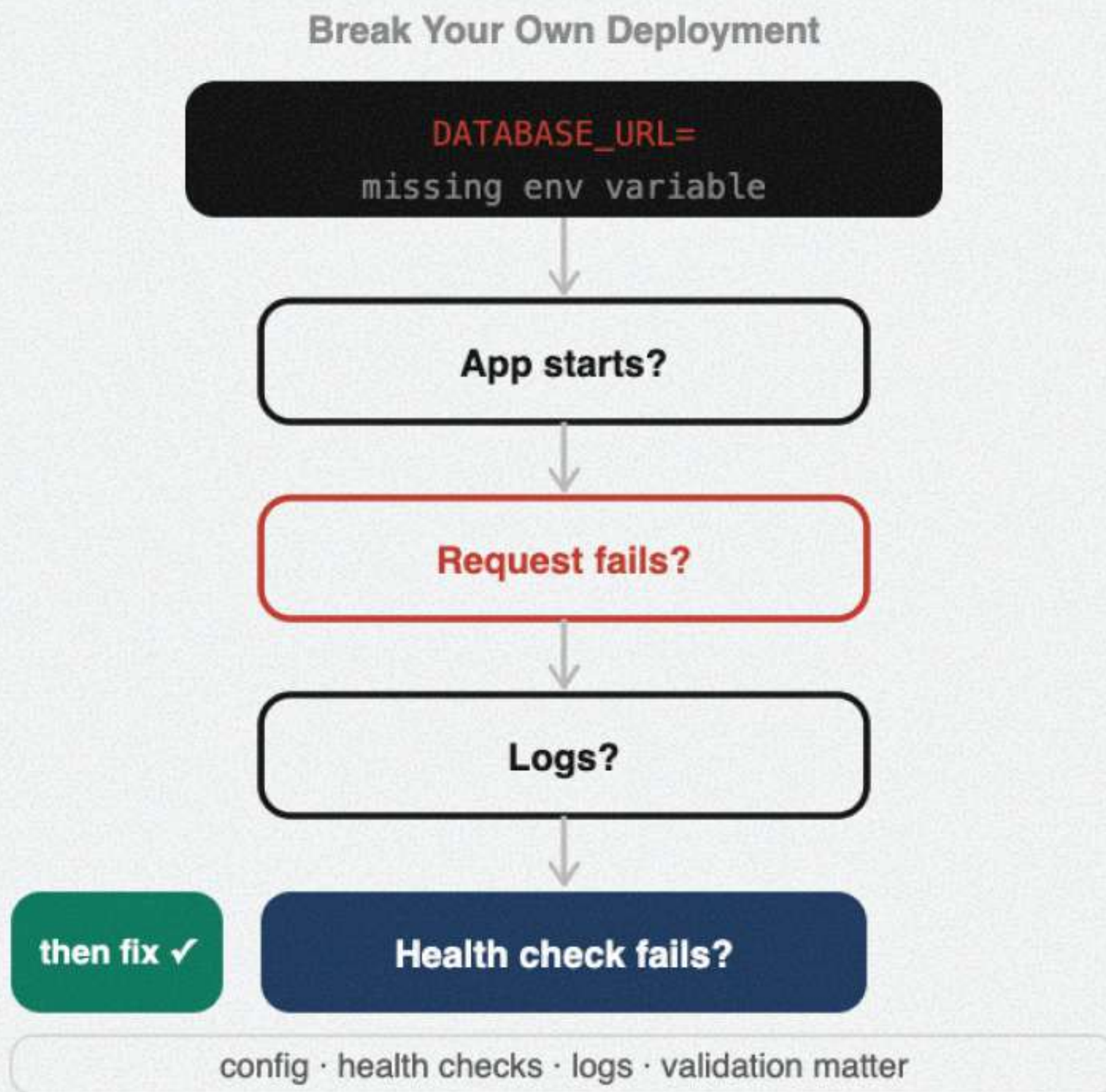
- More workers
- Rate limiting
- Backpressure

You'll understand

Why systems need ways to control overwhelming workloads.

Break Your Own Deployment

In a disposable environment, deploy a version with an intentional configuration problem.



Then fix it.

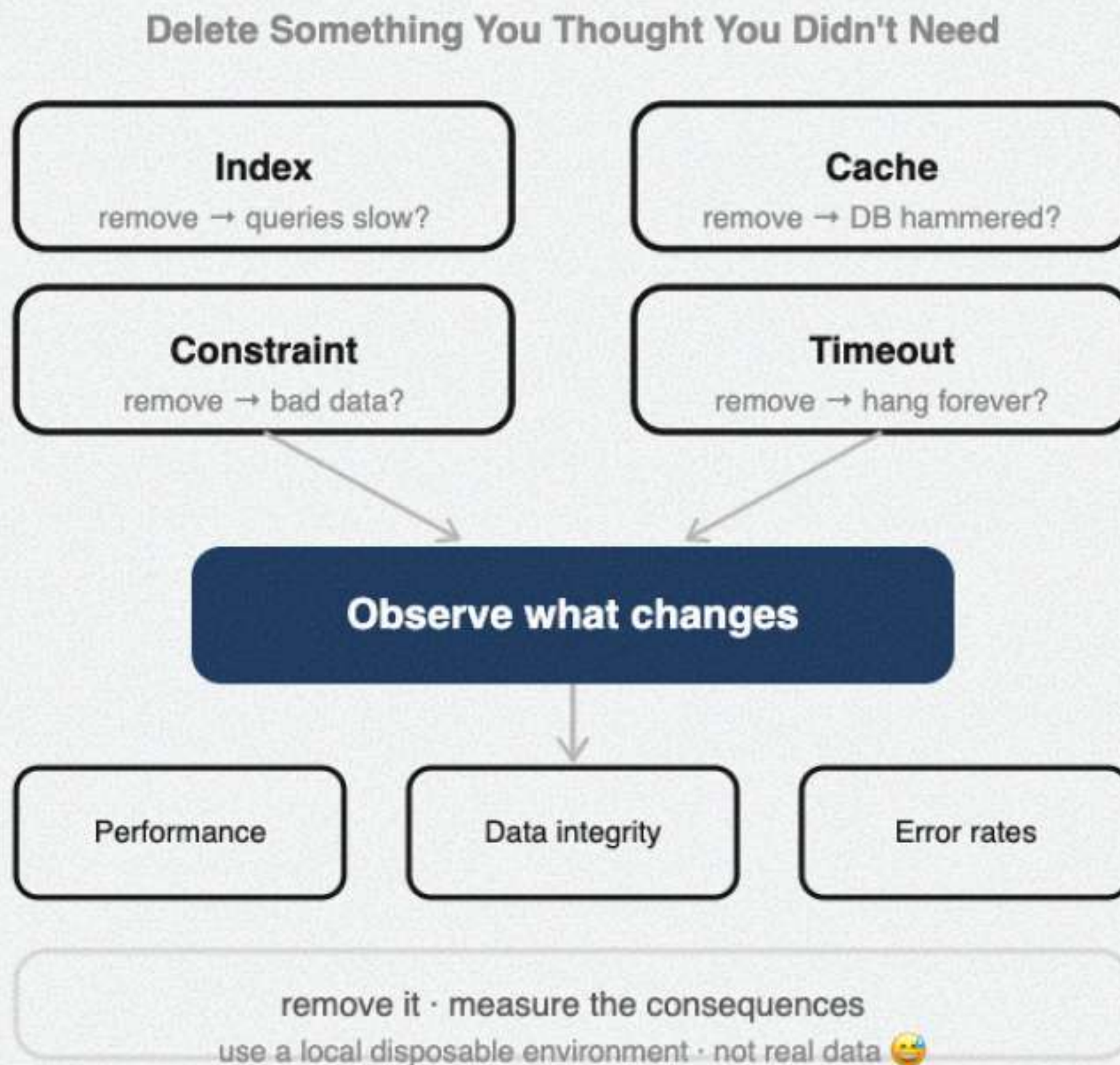
You'll understand

Why configuration, health checks, logs, and deployment validation matter.

Remove a Dependency You Thought You Didn't Need

Not literally your real data. 😭

Use a local disposable database.



You'll understand

Sometimes the best way to understand a system component is to remove it and measure the consequences.

The Bigger Lesson
Don't Just Read About Systems.

Break Them.

Add latency.

Duplicate requests.

Simulate failures.

Create concurrency.

Remove dependencies.

Measure what changes.

**That's how abstract concepts become
things you actually understand.**

***📌 Save this for your next learning
session.***

***💬 Comment "EXPERIMENTS" if you'd try
these.***